

GESTIÓN DE UN GIMNASIO:
([gym-core-system-cli](#))

NOMBRE DEL INTEGRANTE:
Lester Alexander Garcia Felipe

ELABORACIÓN DEL PROYECTO INDIVIDUAL.

DOCENTES INVOLUCRADOS

1.Edwin Salas.
2.Andre Lopez.

CAMPUSLANDS

SALÓN QUETZAL 1

RUTA NODE JS, MYSQL Y EXPRESS

GUATEMALA, GUATEMALA
2026

[gym-core-system-cli](#)

1. SITUACIÓN PROBLEMA

Actualmente muchos entrenadores o gimnasios pequeños gestionan su negocio de forma separada: Algunos utilizan Excel u hojas para la parte financiera o whatsApp para comunicarse con el cliente y enviarle su rutina, dieta y progreso. Además el uso de otra app para la dieta.

La razón crítica de lo financiero y administrativo se debe en no llevar un control de quien esta al día con los pagos y quien no, contratos vigentes, además, el control de gasto por parte del gimnasio(egresos).

Además no hay una trazabilidad sobre si un cliente está cumpliendo sus metas, rutina expirada, o si el plan de alimentación está actualizado.

Podemos deducir que gracias a todo esto y se ha decidido implementar un sistema CLI(Interfaz de Línea de Comandos) o de consola, una solución ligera de bajo consumo de recursos y alta velocidad de ejecución y que no haya una sobrecarga visual o rendimiento de una app web pesada.

Lo que el proyecto busca realizar:

1. Este proyecto busca unificar todo en un solo lugar.
 2. Busca automatizar el control financiero.
 3. Sistematiza alertas y registro de avances.
-

2. LEVANTAMIENTO DE REQUERIMIENTOS

3. REQUERIMIENTOS

3.1. Requerimientos Funcionales

1. Gestión de Clientes

- **RF1.1:** El sistema debe permitir crear, listar, actualizar y eliminar (CRUD) registros de clientes.
- **RF1.2:** El sistema debe permitir asociar un cliente a uno o varios planes de entrenamiento.

2. Gestión de Planes de Entrenamiento

- **RF2.1:** El sistema debe permitir crear planes de entrenamiento con nombre, duración, metas físicas y nivel (principiante, intermedio, avanzado).
- **RF2.2:** El sistema debe permitir asociar planes de entrenamiento a uno o varios clientes.
- **RF2.3:** El sistema debe permitir renovar, cancelar o dar por finalizado un plan de entrenamiento activo.

3. Contratos automáticos

- **RF3.1:** El sistema debe **generar de forma automática un contrato** en el momento exacto en que se asigna un plan a un cliente.
- **RF3.2:** El contrato generado debe incluir obligatoriamente: condiciones, duración, precio, fecha de inicio y fecha de fin.
- **RF3.3:** El sistema debe vincular directamente el contrato con el cliente y el plan correspondiente.

4. Seguimiento Físico (Progreso)

- **RF4.1:** El sistema debe registrar avances semanales del cliente que incluyan: peso, grasa corporal, medidas, fotos y comentarios.
- **RF4.2:** El sistema debe mostrar de forma visual un historial cronológico del progreso físico del cliente.
- **RF4.3:** El sistema debe permitir eliminar registros de avance físico.

5. Nutrición y Alimentación

- **RF5.1:** El sistema debe permitir crear planes de alimentación asociados directamente a un cliente y a su plan de entrenamiento actual.
- **RF5.2:** El sistema debe registrar los alimentos diarios consumidos junto con sus calorías estimadas.
- **RF5.3:** El sistema debe generar y consultar un reporte nutricional semanal de cada cliente.

6. Gestión Financiera

- **RF6.1:** El sistema debe registrar ingresos monetarios procedentes de mensualidades o sesiones individuales.
 - **RF6.2:** El sistema debe registrar egresos monetarios derivados de servicios, compra de suplementos o gastos operativos.
 - **RF6.3:** El sistema debe permitir consultar el balance financiero filtrado por rango de fechas o de forma específica por cliente.
-

3.2. Requerimientos No Funcionales

1. Arquitectura y Paradigma de Desarrollo

- **RNF1.1 (Paradigma):** La aplicación debe estar completamente desarrollada bajo el paradigma de **Programación Orientada a Objetos (POO)**.
- **RNF1.2 (Principios):** El código debe aplicar rigurosamente los principios **SOLID**.
- **RNF1.3 (Patrones):** Se deben implementar de forma evidente al menos **dos patrones de diseño** (ej. *Repository*, *Factory*, *Command*, o *Observer*).
- **RNF1.4 (Estructura):** El proyecto debe organizarse estrictamente en directorios específicos: `/models`, `/services`, `/commands`, `/config`, `/utils`.

2. Interfaz de Usuario (UI) y Entorno

- **RNF2.1 (Entorno):** Debe ser una **aplicación de línea de comandos (CLI)** que se ejecute nativamente en Node.js.
- **RNF2.2 (Experiencia en Consola):** La experiencia interactiva debe potenciarse utilizando librerías específicas de npm como `inquirer` (para menús y formularios), `chalk` (para estilos y colores) y `dotenv` (para variables de entorno).

3. Base de Datos y Consistencia (Crítica)

- **RNF3.1 (Tecnología):** Los datos se deben persistir en **MySQL**
- **RNF3.2 (Conector):** Se debe utilizar obligatoriamente el **driver oficial de mysql2**.
- **RNF3.3 (Transaccionalidad):** Las operaciones de pago y lógica crítica deben usar **transacciones reales de MySQL**.
- **RNF3.4 (Consistencia y Rollback):** Si un cliente cancela un plan de entrenamiento, el sistema debe ejecutar un *rollback* automático que elimine/revierta de forma consistente su seguimiento físico y su contrato asociado.

4. Gestión y Documentación del Proyecto

4. **RNF4.1 (Metodología):** El proyecto debe planificarse y ejecutarse bajo el marco de trabajo **SCRUM** (roles definidos, planeación de Sprints e Historias de Usuario).
5. **RNF4.2 (Control de Versiones):** Los commits del repositorio de GitHub deben seguir obligatoriamente el estándar **Conventional Commits** (ej: `feat:`, `fix:`).
6. **RNF4.3 (Validación de Datos):** La carpeta `/models` debe contener objetos de JavaScript que apliquen validaciones estrictas por campo (tipos de datos, campos requeridos, formatos, rangos)

7. HISTORIAS DE USUARIO CON CRITERIOS DE ACEPTACIÓN

Requerimientos funcionales.

[RF] - RF1.1: Gestión de Clientes (CRUD)

Prioridad:	Alta
Actor:	Administrador
Descripción:	Como Administrador, quiero poder crear, listar, actualizar y eliminar registros de clientes en el sistema para mantener un control actualizado de los miembros del gimnasio.
Funcionalidad:	Permitir al administrador interactuar con una interfaz CLI interactiva para capturar datos de clientes (Nombre, Edad, Correo), guardarlos en la base de datos MySQL,

	listarlos en una tabla limpia, modificar sus datos existentes o borrarlos lógicamente.
Criterios de Aceptación:	1. El sistema debe permitir que el administrador registre un cliente validando que el correo no esté duplicado. 2. El sistema debe permitir que el administrador liste todos los clientes con formato legible por consola. 3. El sistema debe permitir actualizar los datos individuales de un cliente de forma inmediata.
Restricciones:	El sistema no debe permitir la eliminación física si existen dependencias financieras activas sin confirmación previa.

[RF] - RF1.2: Asociación de Clientes a Planes

Prioridad:	Alta
Actor:	Administrador
Descripción:	Como Administrador, quiero poder asociar un cliente a uno o varios planes de entrenamiento para formalizar su inscripción.
Funcionalidad:	Vincular el ID del cliente con uno o múltiples registros de planes en la base de datos a través de menús desplegables controlados por la librería Inquirer.
Criterios de Aceptación:	1. El sistema debe permitir que el administrador seleccione un cliente registrado. 2. El sistema debe permitir asociar un plan de entrenamiento vigente al cliente seleccionado.
Restricciones:	El cliente debe existir previamente en la base de datos para poder asignarle un plan.

[RF] - RF2.1: Creación de Planes de Entrenamiento

Prioridad:	Alta
Actor:	Administrador
Descripción:	Como Administrador, puedo crear nuevos planes de entrenamiento especificando sus atributos físicos principales para diversificar la oferta del establecimiento.
Funcionalidad:	Formulario de consola que capture: Nombre del plan, duración en semanas/meses, metas físicas y nivel de dificultad.
Criterios de Aceptación:	1. El sistema debe permitir que el administrador defina el nivel únicamente entre las opciones: principiante, intermedio

	y avanzado. 2. El sistema debe registrar la meta física obligatoriamente.
Restricciones:	La duración del plan no puede ser igual o menor a cero.

[RF] - RF2.2: Asociación Múltiple de Planes

Prioridad:	Media
Actor:	Administrador
Descripción:	Como Administrador, puedo vincular planes de entrenamiento a múltiples usuarios de forma masiva o individual.
Funcionalidad:	Procesamiento de mapeos relacionales en MySQL para permitir la asignación N a N de planes y clientes.
Criterios de Aceptación:	1. El sistema debe permitir que un plan sea asignado a varios clientes al mismo tiempo. 2. El sistema debe actualizar el estado de los planes de cada cliente afectado.
Restricciones:	Los planes deben estar en estado Activo en el catálogo para ser elegibles.

[RF] - RF2.3: Ciclo de Vida del Plan (Renovar/Cancelar/Finalizar)

Prioridad:	Alta
Actor:	Administrador
Descripción:	Como Administrador, puedo gestionar el estado de un plan activo para reflejar renovaciones, cancelaciones o culminaciones del servicio.
Funcionalidad:	Cambio de estado en el registro de la suscripción del cliente disparando acciones asociadas en cascada.
Criterios de Aceptación:	1. El sistema debe permitir cambiar el estado de un plan activo a 'Cancelado' o 'Finalizado'. 2. El sistema debe actualizar automáticamente la fecha de fin al renovar un plan.
Restricciones:	Un plan finalizado no puede ser cancelado retrospectivamente.

[RF] - RF3.1: Generación Automática de Contratos

Prioridad:	Alta
Actor:	Sistema
Descripción:	Como Sistema, debo generar automáticamente un contrato digital en el

	momento exacto en que se asigna un plan para garantizar la formalidad legal.
Funcionalidad:	Triggers automáticos a nivel de capa de servicio que instancien y guarden un objeto Contrato en la BD.
Criterios de Aceptación:	1. El sistema debe generar de forma automática un contrato en el momento exacto en que se asigna un plan a un cliente.
Restricciones:	La transacción se cancelará por completo si el contrato no se genera correctamente.

[RF] - RF3.2: Contenido Obligatorio del Contrato

Prioridad:	Alta
Actor:	Sistema
Descripción:	Como Sistema, garantizo que todo contrato incluya los términos comerciales explícitos acordados.
Funcionalidad:	Validación estricta de campos obligatorios en el modelo de persistencia del contrato.
Criterios de Aceptación:	1. El contrato generado debe incluir obligatoriamente: condiciones, duración, precio, fecha de inicio y fecha de fin.
Restricciones:	No se permite la creación de un contrato con campos de precio nulos o fechas inconsistentes.

[RF] - RF3.3: Vinculación Directa Relacional del Contrato

Prioridad:	Alta
Actor:	Sistema
Descripción:	Como Sistema, mantengo la integridad de datos vinculando el contrato con sus entidades relacionadas.
Funcionalidad:	Asignación de llaves foráneas indexadas en la base de datos de MySQL para conectar Contrato, Cliente y Plan.
Criterios de Aceptación:	1. El sistema debe vincular directamente el contrato con el cliente y el plan correspondiente de forma inequívoca.
Restricciones:	Ningún contrato puede quedar huérfano (sin cliente o plan asociado).

[RF] - RF4.1: Registro de Avances Semanales

Prioridad:	Media
Actor:	Administrador / Entrenador
Descripción:	Como Entrenador, puedo registrar las métricas físicas semanales del cliente para

	monitorear el cumplimiento de sus objetivos.
Funcionalidad:	Formulario estructurado de inserción de datos antropométricos y multimedia (rutas de fotos).
Criterios de Aceptación:	1. El sistema debe registrar avances semanales del cliente que incluyan: peso, grasa corporal, medidas, fotos y comentarios.
Restricciones:	El porcentaje de grasa corporal debe expresarse en un rango de 1 a 100.

[RF] - RF4.2: Historial Cronológico de Progreso

Prioridad:	Media
Actor:	Administrador
Descripción:	Como Administrador, puedo visualizar el histórico de progreso del cliente para evaluar su evolución temporal.
Funcionalidad:	Impresión de gráficas ASCII o tablas ordenadas cronológicamente en la consola de Node.js.
Criterios de Aceptación:	1. El sistema debe mostrar de forma visual un historial cronológico del progreso físico del cliente.
Restricciones:	Los registros deben ordenarse obligatoriamente de la fecha más antigua a la más reciente.

[RF] - RF4.3: Eliminación de Avances Físicos

Prioridad:	Baja
Actor:	Administrador
Descripción:	Como Administrador, puedo eliminar un registro erróneo de avance físico para depurar la bitácora del cliente.
Funcionalidad:	Operación de borrado directo sobre un ID específico del avance.
Criterios de Aceptación:	1. El sistema debe permitir eliminar registros de avance físico de manera permanente.
Restricciones:	Esta acción requiere confirmación doble del Administrador en la consola.

[RF] - RF5.1: Creación de Planes Alimenticios Coherentes

Prioridad:	Alta
Actor:	Administrador /Entrenador

Descripción:	Como Entrenador, puedo estructurar planes nutricionales asociados directamente al perfil deportivo del cliente.
Funcionalidad:	Vinculación lógica entre las tablas de Nutrición, Clientes y Planes de Entrenamiento.
Criterios de Aceptación:	1. El sistema debe permitir crear planes de alimentación asociados directamente a un cliente y a su plan de entrenamiento actual.
Restricciones:	El plan de alimentación debe coincidir temporalmente con la vigencia del plan de entrenamiento.

[RF] - RF5.2: Registro Diario de Alimentos y Calorías

Prioridad:	Media
Actor:	Entrenador
Descripción:	Como Entrenador, puedo ingresar la ingesta diaria del cliente para controlar el balance energético.
Funcionalidad:	Módulo de bitácora diaria que calcula sumatorias de calorías.
Criterios de Aceptación:	1. El sistema debe registrar los alimentos diarios consumidos junto con sus calorías estimadas de forma acumulativa.
Restricciones:	El valor de las calorías no puede ser un número negativo.

[RF] - RF5.3: Reporte Nutricional Semanal

Prioridad:	Media
Actor:	Entrenador
Descripción:	Como Entrenador, puedo consultar el consolidado nutricional de la semana para tomar decisiones sobre la dieta.
Funcionalidad:	Generación de resúmenes agregados con cálculos promedio por pantalla.
Criterios de Aceptación:	1. El sistema debe generar y consultar un reporte nutricional semanal de cada cliente.
Restricciones:	Requiere que existan al menos 3 días de registros diarios en esa semana para procesar el reporte.

[RF] - RF6.1: Registro de Ingresos Financieros

Prioridad:	Alta
Actor:	Administrador
Descripción:	Como Administrador, puedo registrar los ingresos monetarios para mantener la contabilidad exacta de la empresa.

Funcionalidad:	Registro en la tabla contable indicando monto, concepto (mensualidad/sesión) y marca de tiempo.
Criterios de Aceptación:	1. El sistema debe registrar ingresos monetarios procedentes de mensualidades o sesiones individuales.
Restricciones:	El monto del ingreso debe ser mayor a cero.

[RF] - RF6.2: Registro de Egresos Operativos

Prioridad:	Alta
Actor:	Administrador
Descripción:	Como Administrador, puedo asentar los gastos del negocio para visualizar las salidas de dinero.
Funcionalidad:	Captura contable de salidas por servicios, suplementos y costos de operación.
Criterios de Aceptación:	1. El sistema debe registrar egresos monetarios derivados de servicios, compra de suplementos o gastos operativos.
Restricciones:	Cada egreso debe tener un comprobante o descripción obligatoria.

[RF] - RF6.3: Consulta de Balance Financiero Filtrado.

Prioridad:	Alta
Actor:	Administrador
Descripción:	Como Administrador, puedo consultar el estado financiero global o específico para evaluar la rentabilidad.
Funcionalidad:	Consultas SQL dinámicas con cláusulas WHERE avanzadas para rangos de fechas o IDs de clientes.
Criterios de Aceptación:	1. El sistema debe permitir consultar el balance financiero filtrado por rango de fechas o de forma específica por cliente.
Restricciones:	La fecha inicial del filtro no puede ser posterior a la fecha final.

Requerimientos no funcionales.

[RNF] - RNF1.1: Paradigma de Programación Orientada a Objetos

Prioridad:	Alta
Actor:	Sistema / Desarrollador
Descripción:	La aplicación de software debe estar completamente diseñada y estructurada bajo el enfoque del paradigma de Programación Orientada a Objetos (POO).

Funcionalidad:	Uso estricto de clases de JavaScript (ES6+), encapsulamiento de atributos, herencia cuando sea pertinente y abstracción clara de las entidades de negocio.
Criterios de Aceptación:	1. La aplicación debe estar completamente desarrollada bajo el paradigma de Programación Orientada a Objetos (POO). 2. No se permiten funciones sueltas o scripts estructurados fuera de un contexto de clase organizativo.
Restricciones:	Exclusivo uso del entorno runtime Node.js con soporte moderno para clases.

[RNF] - RNF1.2: Aplicación de Principios SOLID

Prioridad:	Alta
Actor:	Sistema / Desarrollador
Descripción:	El diseño de la arquitectura del código debe respetar y aplicar de forma rigurosa los cinco principios SOLID para garantizar mantenibilidad.
Funcionalidad:	Separación de responsabilidades en clases únicas, inyección de dependencias para evitar acoplamiento rígido e interfaces/clases extensibles pero cerradas a modificación.
Criterios de Aceptación:	1. El código debe aplicar rigurosamente los principios SOLID. 2. Cada clase del directorio /services o /commands debe poseer una única responsabilidad bien delimitada.
Restricciones:	Cualquier violación detectada en revisiones de arquitectura invalidará la aprobación del entregable.

[RNF] - RNF1.3: Implementación de Patrones de Diseño

Prioridad:	Alta
Actor:	Sistema / Desarrollador
Descripción:	El sistema debe utilizar al menos dos patrones de diseño de software reconocidos para resolver problemas de instanciación o flujo de datos.
Funcionalidad:	Implementación explícita de patrones como Repository (para el aislamiento de la base de datos) y Factory o Command (para encapsular las acciones de la CLI).
Criterios de Aceptación:	1. Se deben implementar de forma evidente al menos dos patrones de diseño (ej. Repository, Factory, Command, o Observer).

Restricciones:	Los patrones implementados deben estar documentados dentro de la estructura de comentarios del código.
-----------------------	--

[RNF] - RNF1.4: Estructura Estricta de Directorios

Prioridad:	Alta
Actor:	Sistema / Desarrollador
Descripción:	La organización física del árbol del código fuente debe apegarse estrictamente a un estándar de arquitectura limpio.
Funcionalidad:	Segmentación completa del proyecto en las rutas asignadas por el estándar arquitectónico.
Criterios de Aceptación:	1. El proyecto debe organizarse estrictamente en directorios específicos: /models, /services, /commands, /config, /utils.
Restricciones:	No se permite la creación de archivos de lógica de negocio en la raíz del proyecto.

[RNF] - RNF2.1: Interfaz de Línea de Comandos (CLI) Nativa

Prioridad:	Alta
Actor:	Sistema
Descripción:	El entorno de ejecución e interacción para el usuario final debe ser directamente la consola mediante una arquitectura de línea de comandos fluida.
Funcionalidad:	Ejecución por comandos de consola nativos soportados directamente por Node.js.
Criterios de Aceptación:	1. Debe ser una aplicación de línea de comandos (CLI) que se ejecute nativamente en Node.js.
Restricciones:	No se debe desarrollar ninguna interfaz gráfica web (HTML/CSS) o móvil para esta entrega.

[RNF] - RNF2.2: Experiencia en Consola Interactiva Avanzada

Prioridad:	Media
Actor:	Sistema
Descripción:	La interfaz de consola debe brindar una experiencia interactiva rica, intuitiva y visualmente clara mediante componentes específicos.
Funcionalidad:	Integración y configuración de librerías oficiales del ecosistema npm para menús y validación visual.

Criterios de Aceptación:	1. La experiencia interactiva debe potenciarse utilizando librerías específicas de npm como inquirer (para menús y formularios), chalk (para estilos y colores) y dotenv (para variables de entorno).
Restricciones:	Todas las capturas de datos complejas deben realizarse a través de los prompts controlados de Inquirer.

[RNF] - RNF3.1: Persistencia de Datos en Servidor MySQL

Prioridad:	Alta
Actor:	Sistema
Descripción:	Todos los datos operacionales de los clientes, planes, contratos y finanzas deben ser almacenados permanentemente en un motor de bases de datos relacional.
Funcionalidad:	Modelado de base de datos relacional con tablas normalizadas mapeadas en una instancia activa de MySQL.
Criterios de Aceptación:	1. Los datos se deben persistir de forma obligatoria en el motor relacional MySQL.
Restricciones:	No se permite el uso de almacenamiento local en archivos JSON o bases de datos NoSQL.

[RNF] - RNF3.2: Uso Obligatorio de Conector mysql2

Prioridad:	Alta
Actor:	Sistema
Descripción:	La comunicación de bajo nivel entre la aplicación Node.js y el servidor de base de datos MySQL debe ejecutarse con el driver oficial.
Funcionalidad:	Configuración de un pool de conexiones optimizado utilizando exclusivamente el paquete npm mysql2.
Criterios de Aceptación:	1. Se debe utilizar obligatoriamente el driver oficial de mysql2 para ejecutar las consultas.
Restricciones:	Queda prohibido el uso de ORMs pesados (como Sequelize o TypeORM) para asegurar el dominio de consultas nativas.

[RNF] - RNF3.3: Transaccionalidad en Operaciones Críticas

Prioridad:	Alta
Actor:	Sistema
Descripción:	Las operaciones que involucren flujos financieros o lógica multi-tabla deben

	cumplir estrictamente con las propiedades ACID mediante transacciones explícitas.
Funcionalidad:	Implementación de bloques START TRANSACTION, COMMIT y ROLLBACK nativos sobre el pool de conexiones de mysql2.
Criterios de Aceptación:	1. Las operaciones de pago y lógica crítica deben usar transacciones reales de MySQL.
Restricciones:	Si un paso intermedio falla, la base de datos no debe guardar ningún cambio parcial.

[RNF] - RNF3.4: Consistencia Crítica y Rollback en Cascada

Prioridad:	Alta
Actor:	Sistema
Descripción:	El sistema debe asegurar la reversión total y consistente de los datos asociados ante eventos drásticos como la cancelación fulminante de un plan de entrenamiento.
Funcionalidad:	Mecanismo automatizado controlado que ejecute un rollback integral borrando o revirtiendo con consistencia absoluta las dependencias lógicas inmediatas.
Criterios de Aceptación:	1. Si un cliente cancela un plan de entrenamiento, el sistema debe ejecutar un rollback automático que elimine/revierta de forma consistente su seguimiento físico y su contrato asociado.
Restricciones:	No pueden quedar registros huérfanos de contratos o seguimientos físicos tras una cancelación.

[RNF] - RNF4.1: Gestión del Proyecto bajo Marco Trabajo SCRUM

Prioridad:	Media
Actor:	Equipo de Desarrollo
Descripción:	La planeación, seguimiento y control del ciclo de vida del desarrollo de software debe apegarse a los ritos ágiles de la metodología SCRUM.
Funcionalidad:	Planificación orientada a objetivos mediante Sprints acotados, asignación de roles técnicos y diseño de Historias de Usuario claras.
Criterios de Aceptación:	1. El proyecto debe planificarse y ejecutarse bajo el marco de trabajo SCRUM (roles definidos, planeación de Sprints e Historias de Usuario).
Restricciones:	Cada funcionalidad añadida debe estar justificada por una tarjeta o HU activa en el Product Backlog.

[RNF] - RNF4.2: Control de Versiones con Conventional Commits

Prioridad:	Media
Actor:	Equipo de Desarrollo
Descripción:	El historial del repositorio Git debe ser semántico, limpio y fácilmente legible por herramientas de automatización.
Funcionalidad:	Aplicación estricta de prefijos y reglas de formateo estándar en los mensajes de confirmación de cambios (commits).
Criterios de Aceptación:	1. Los commits del repositorio de GitHub deben seguir obligatoriamente el estándar Conventional Commits (ej: feat:, fix:, docs:, chore:).
Restricciones:	Los commits con mensajes genéricos o descriptivos ambiguos (ej. 'cambios', 'fix') no serán aceptados en la rama principal.

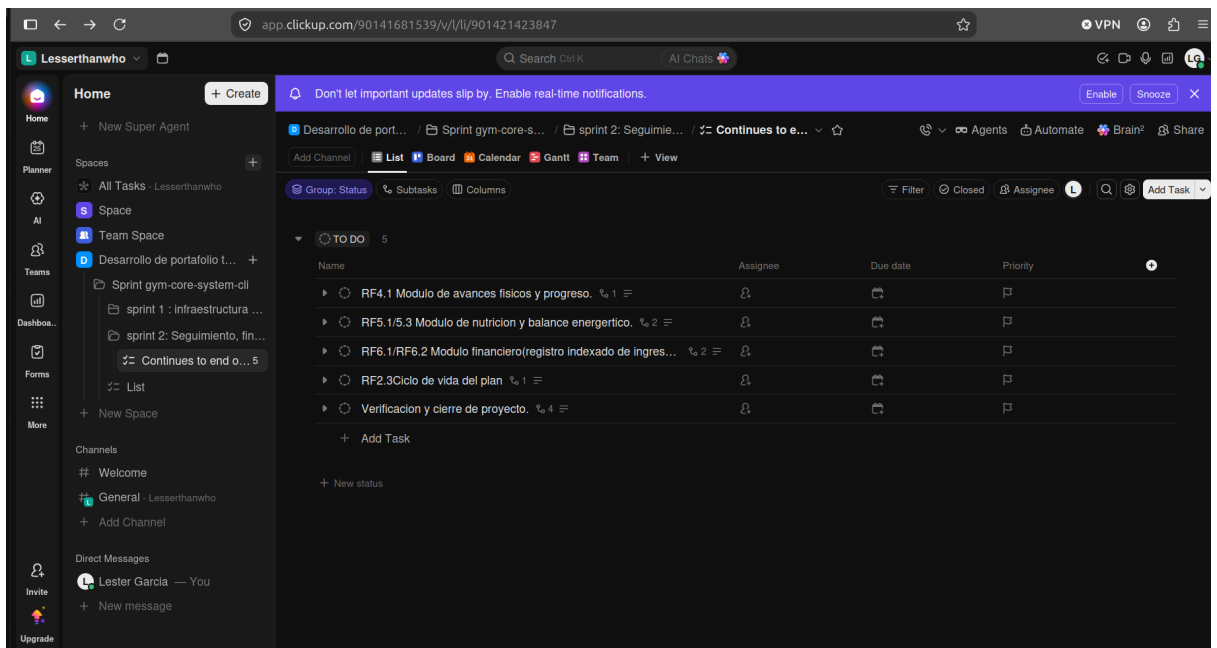
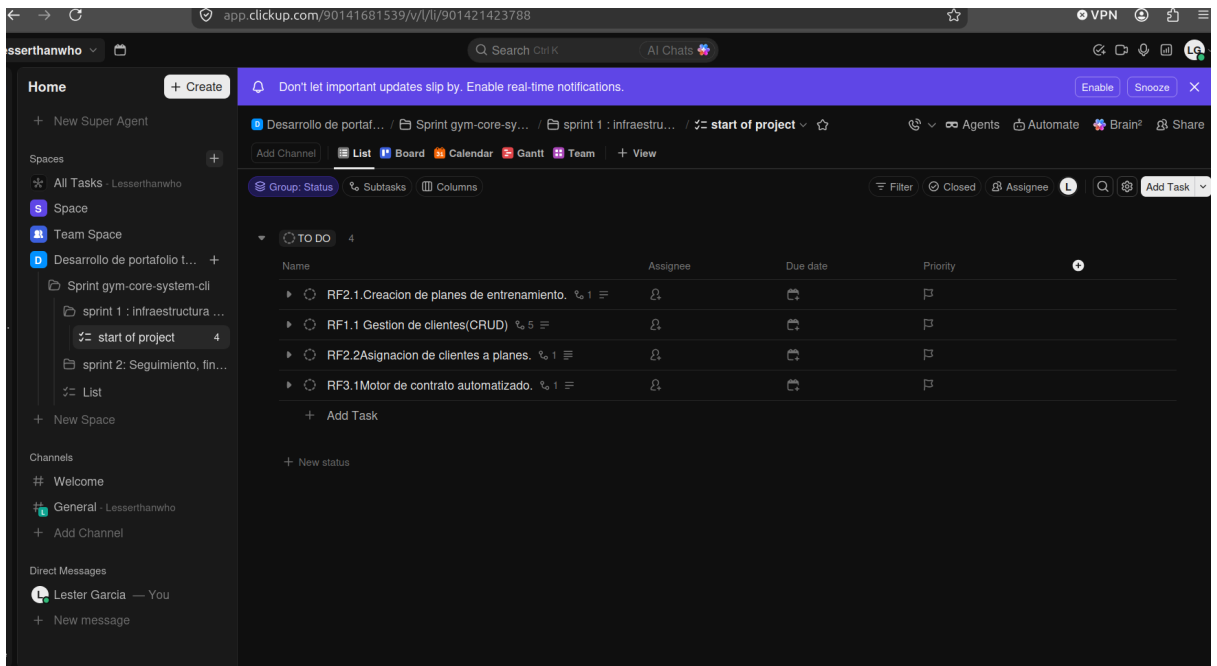
[RNF] - RNF4.3: Validación Estricta de Datos en Capa /models

Prioridad:	Alta
Actor:	Sistema
Descripción:	Los objetos e instancias de negocio que representen tablas de la BD deben encargarse de sanitizar y validar los datos antes de cualquier procesamiento.
Funcionalidad:	Lógica interna dentro de los constructores o métodos validadores de los modelos para chequear tipos, longitudes y formatos.
Criterios de Aceptación:	1. La carpeta /models debe contener objetos de JavaScript que apliquen validaciones estrictas por campo (tipos de datos, campos requeridos, formatos, rangos).
Restricciones:	Ningún dato erróneo o mal formateado debe llegar a la capa de servicios o repositorios.

1. METODOLOGÍA

La metodología implementada en este proceso fue SCRUM. Por lo que durante el proceso se crearon dos sprints y cada uno con diferentes tareas asignadas a realizar por día a cada integrante del equipo aunque en este caso solo fue individual.

2. EVIDENCIA DE PLANTEAMIENTO DE PLATAFORMA DE TRABAJO.



Home

+ New Super Agent

Planner

All Tasks - Lesserthanwho

Space

Team Space

Desarrollo de portafolio t...

Sprint gym-core-system-cli

sprint 1 : Infraestructura ...

start of project 4

sprint 2: Seguimiento, fin...

List

New Space

Channels

Welcome

Don't let important updates slip by. Enable real-time notifications.

Enable Snooze X

Desarrollo de portaf... / Sprint gym-core-sy... / sprint 1 : Infraestructu... / start of project

Add Channel | List Board Calendar Gantt Team + View

Group: Status Subtasks Columns

Filter Closed Assignee L Q Add Task

IN PROGRESS 1

Name	Assignee	Due date	Priority
RF1.1 Gestion de clientes(CRUD) % 5			
Configuracion inicial de entorno			
Creacion de base de datos y modelo fisico.			
Conexion con mysql.			
Implementacion de operaciones CRUD.			
Disenio de menu interactivo con inquire y chalk.			
+ Add Task			

Home

+ New Super Agent

Planner

All Tasks - Lesserthanwho

Space

Team Space

Desarrollo de portafolio t...

Sprint gym-core-system-cli

sprint 1 : Infraestructura ...

start of project 4

sprint 2: Seguimiento, fin...

List

New Space

Channels

Welcome

Don't let important updates slip by. Enable real-time notifications.

Enable Snooze X

Desarrollo de portaf... / Sprint gym-core-sy... / sprint 1 : Infraestructu... / start of project

Add Channel | List Board Calendar Gantt Team + View

Group: Status Subtasks Columns

Filter Closed Assignee L Q Add Task

IN PROGRESS 1

Name	Assignee	Due date	Priority
RF1.1 Gestion de clientes(CRUD) % 5			
Configuracion inicial de entorno			
Creacion de base de datos y modelo fisico.			
Conexion con mysql.			
Implementacion de operaciones CRUD.			
Disenio de menu interactivo con inquire y chalk.			
+ Add Task			

Don't let important updates slip by. Enable real-time notifications.

Enable Snooze X

Desarrollo de portaf... / Sprint gym-core-sy... / sprint 1 : Infraestructu... / start of project

Add Channel | Board Calendar Gantt Team List + View

Group: Status Subtasks Columns

Filter Closed Assignee L Q Add Task

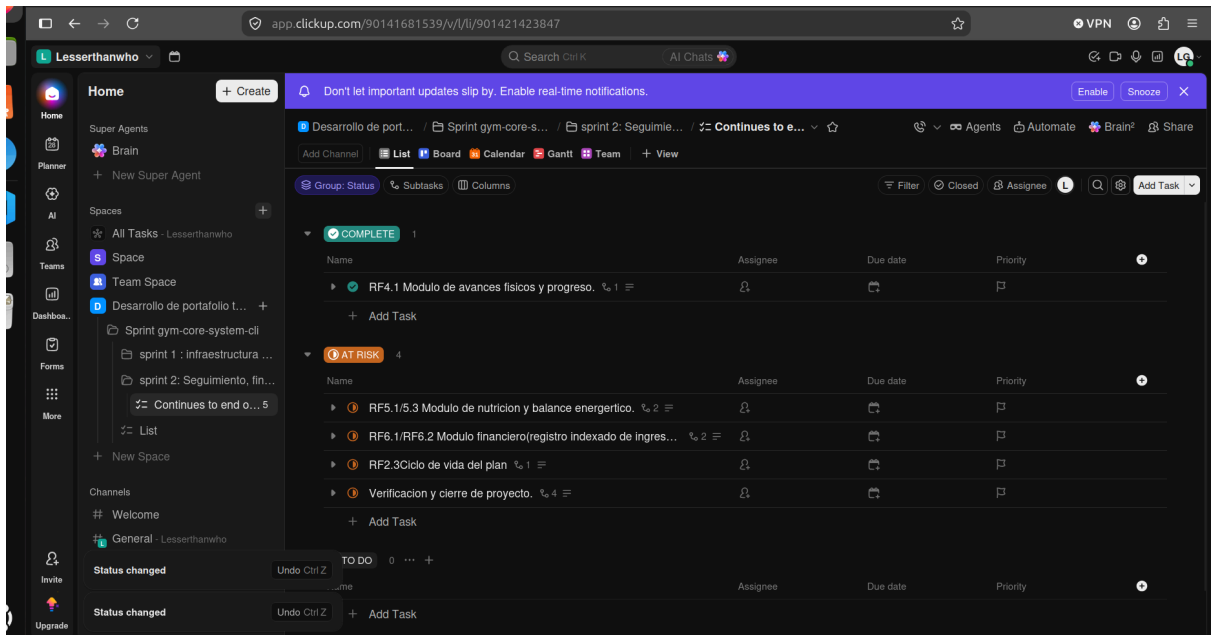
COMPLETE 4 ... +

Name	Assignee	Due date	Priority
RF1.1 Gestion de clientes(CRUD) % 5			
RF2.1.Creacion de planes de entrenamiento. % 1			
RF2.2Asignacion de clientes a planes. % 1			
RF3.1Motor de contrato automatizado. % 1			
+ Add Task			

TO DO 0

Name	Assignee	Due date	Priority
+ Add Task			

+ New status



7. CONCLUSIONES

Trabajar con esta metodología ágil incrementa la organización y las buenas practicas para elaborar un proyecto de semejante escala como este..